Implement an 8-puzzle problem using

i. DFS

```
def get_neighbors(state):
    zero = state.index(0)
    row, col = divmod(zero, 3)
    neighbors = []

    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        r, c = row + dr, col + dc

        if 0 <= r < 3 and 0 <= c < 3:
            new_zero = r * 3 + c
            temp = list(state)
            temp[zero], temp[new_zero] = temp[new_zero], temp[zero]
            neighbors.append(tuple(temp))

    return neighbors


def dfs(start, goal):
    stack = [(start, [start])]
    visited = set()

    while stack:
        state, path = stack.pop()

        if state == goal:
            return path

        if state in visited:
            continue

        visited.add(state)

        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                stack.append((neighbor, path + [neighbor]))

    return None
```

```
def display(path):
    if path is None:
        print("No solution found")
        return

    print("Solution found in", len(path) - 1, "moves")

    for state in path:
        for i in range(0, 9, 3):
            print(state[i:i + 3])
        print()


start = (1, 2, 3,
        7, 8, 4,
        6, 0, 5)

goal = (1, 2, 3,
        8, 0, 4,
        7, 6, 5)

display(dfs(start, goal))
```

OUTPUT:

```
PS C:\Users\BMSCE\Desktop\1BM24CS327> & C:
7/2DFS.py
Solution found in 27 moves
(1, 2, 3)
(7, 8, 4)
(6, 0, 5)

(1, 2, 3)
(7, 8, 4)
(6, 5, 0)

(1, 2, 3)
(7, 8, 0)
(6, 5, 4)

(1, 2, 3)
(7, 0, 8)
(6, 5, 4)

(1, 2, 3)
(0, 7, 8)
(6, 5, 4)

(1, 2, 3)
(6, 7, 8)
(0, 5, 4)

(1, 2, 3)
(6, 7, 8)
(5, 0, 4)

(1, 2, 3)
(6, 7, 8)
(5, 4, 0)

(1, 2, 3)
(6, 7, 0)
(5, 4, 8)

(1, 2, 3)
(6, 0, 7)
(5, 4, 8)

(1, 2, 3)
(0, 6, 7)
(5, 4, 8)

(1, 2, 3)
(5, 6, 7)
(0, 4, 8)

(1, 2, 3)
(5, 6, 7)
(4, 0, 8)

(1, 2, 3)
(5, 6, 7)
(4, 8, 0)

(1, 2, 3)
(5, 6, 0)
(4, 8, 7)

(1, 2, 3)
(5, 0, 6)
(4, 8, 7)

(1, 2, 3)
(0, 5, 6)
(4, 8, 7)

(1, 2, 3)
(4, 5, 6)
(0, 8, 7)

(1, 2, 3)
(4, 5, 6)
(8, 0, 7)

(1, 2, 3)
(4, 5, 6)
(8, 7, 0)

(1, 2, 3)
(4, 5, 0)
(8, 7, 6)

(1, 2, 3)
(4, 0, 5)
(8, 7, 6)

(1, 2, 3)
(0, 4, 5)
(8, 7, 6)
```

```
(1, 2, 3)
(8, 4, 5)
(0, 7, 6)

(1, 2, 3)
(8, 4, 5)
(7, 0, 6)

(1, 2, 3)
(8, 4, 5)
(7, 6, 0)

(1, 2, 3)
(8, 4, 0)
(7, 6, 5)

(1, 2, 3)
(8, 0, 4)
(7, 6, 5)
```

Implement an 8-puzzle problem using
ii. Iterative deepening search (IDS) algorithm

```
def get_neighbors(state):
    zero = state.index(0)
    row, col = divmod(zero, 3)
    neighbors = []

    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        r, c = row + dr, col + dc

        if 0 <= r < 3 and 0 <= c < 3:
            new_zero = r * 3 + c
            temp = list(state)
            temp[zero], temp[new_zero] = temp[new_zero], temp[zero]
            neighbors.append(tuple(temp))

    return neighbors


def depth_limited_search(state, goal, limit, path):
    if state == goal:
        return path
```

```
    if limit == 0:
        return None

    for neighbor in get_neighbors(state):
        if neighbor not in path:
            result = depth_limited_search(
                neighbor, goal, limit - 1, path + [neighbor]
            )

            if result is not None:
                return result

    return None


def ids(start, goal, max_depth=30):
    for depth in range(max_depth + 1):
        result = depth_limited_search(start, goal, depth, [start])

        if result is not None:
            return result

    return None


def display(path):
    if path is None:
        print("No solution found within the depth limit")
        return

    print("Solution found in", len(path) - 1, "moves")

    for state in path:
        for i in range(0, 9, 3):
            print(state[i:i + 3])
        print()


start = (2, 8, 3,
```

```
         1, 6, 4,
         7, 0, 5)

goal = (1, 2, 3,
        8, 0, 4,
        7, 6, 5)



display(ids(start, goal))
```

OUTPUT:

```
PS C:\Users\BMSCE\Desktop\1BM24CS327> & C:\Users\BMSCE\AppData\Roaming\
7/2IDS.py
Solution found in 5 moves
(2, 8, 3)
(1, 6, 4)
(7, 0, 5)

(2, 8, 3)
(1, 0, 4)
(7, 6, 5)

(2, 0, 3)
(1, 8, 4)
(7, 6, 5)

(0, 2, 3)
(1, 8, 4)
(7, 6, 5)

(1, 2, 3)
(0, 8, 4)
(7, 6, 5)

(1, 2, 3)
(8, 0, 4)
(7, 6, 5)
```